UNIVERSITY OF MORATUWA

Faculty of Engineering

Registered Module No: CS4203

Improving Blockchain Scalability: Throughput Enhancement in ZK-Rollups

Project Proposal

Date of Submission:

September 24, 2025

Team Members:

Fernando T.H.L: 210167E
Gamage M.S: 210176F
Lishan S.D: 210339J
Perera M.V.D.P.D.S: 210466U

Supervisor:

Dr. Sunimal Rathnayake (University of Moratuwa)

Department of:

Computer Science and Engineering

Contents

1 Introduction

Blockchain technology has emerged as a transformative innovation, attracting attention from academia, industry, and governments alike. It provides a secure, distributed ledger that enables trust among untrusted entities without relying on centralized authorities or intermediaries. Its defining properties, including immutability, transparency, auditability, autonomy, and resilience, position blockchain as one of the most disruptive digital technologies of the past decade [1].

Initially conceived as the foundation of Bitcoin [2], blockchain's applications have rapidly expanded beyond cryptocurrencies. Ethereum [3] introduced programmability through smart contracts, enabling decentralized applications (DApps) that support diverse use cases. Today, blockchain is being deployed in finance, insurance, supply chain management, healthcare, identity management, registry services, stock trading, the Internet of Things (IoT), and energy systems [1].

Industry projections highlight this momentum. Deloitte's 2019 survey reported that 86% of enterprises anticipated blockchain's mainstream adoption, with 81% planning integration into their systems. Gartner forecasted blockchain's business value to surpass $176 billion by 2025 and $3.1 trillion by 2030, while Cisco predicted that 10% of global GDP would rely on blockchain by 2027 [1]. These estimates underscore blockchain's disruptive potential and reinforce the urgency of addressing its fundamental challenges.

Among these challenges, the surge in adoption has brought to light scalability as the most critical barrier to widespread adoption [4]. The inherent limitation in the number of transactions these networks can process per second has prompted efforts within the blockchain community to develop a wide range of innovative solutions [5, 6].

1.1 Motivation

Public blockchains are constrained by the "scalability trilemma": decentralization, security, and scalability cannot be maximized simultaneously [7]. Ethereum, one of the most prominent public blockchains, exemplifies this trade-off [8].

Ethereum has deliberately optimized for decentralization and security—now under Proof-of-Stake—at the cost of base-layer throughput [3]. In practice, Ethereum L1 capacity remains modest at roughly 15–25 TPS, with real-time trackers often showing about 20 TPS [9,10]. When demand spikes, finite blockspace leads to congestion and fee volatility; as of September 2025, the *average* L1 fee typically ranges around $0.40–$0.60 per transaction [11, 12], while daily transaction counts hover around 1.3–1.7 million [13, 14].

To scale without compromising Ethereum's security guarantees, the ecosystem has converged on Layer-2 (L2) *rollups*. Rollups execute transactions off-chain and post succinct data/proofs to Ethereum, thereby inheriting L1 security while reducing L1 computation and bandwidth [15–18]. Within this family, *ZK-Rollups* offer cryptographic validity proofs that enable faster finality than optimistic rollups (which require a challenge window) [17, 18]. Post–EIP-4844 ("Dencun"), user fees on major L2s dropped by roughly one to two orders of magnitude, often to sub-cent or few-cent levels, catalyzing sustained activity growth [19–21].

Table 1 summarizes indicative, *recent* orders of magnitude comparing Ethereum L1 with high-throughput L2s (e.g., Base, Arbitrum). While exact numbers fluctuate daily, L2s routinely process *millions* of transactions per day (e.g., Base exceeded 300M user operations over a 30-day window in mid-2025, i.e., ∼10M/day on average) [22].

Metric	**Ethereum L1 (Sep 2025)**	**Major Ethereum L2s (e.g., Base / Arbitrum)**
Throughput (TPS)	∼15–25 TPS (typical; real-time trackers around ∼20 TPS) [9, 10]	10^2–10^3+ TPS at peak; e.g., Base often 100+ TPS and higher bursts [22, 23]
Avg. user fee (USD)	∼\$0.40–\$0.60 avg per tx (daily average) [11, 12]	Sub-\$0.10 typical post-Dencun; frequently cents or sub-cent levels [19–21]
Daily transactions	∼1.3–1.7M/day [13, 14]	*Millions/day* routinely; e.g., Base ∼10M/day avg over 30 days in mid-2025 [22]

Table 1: Indicative scale comparison (Sep. 2025). Figures vary over time; ranges shown are representative, with examples from public dashboards.

Yet a gap persists between ZK-Rollups' theoretical scalability and their observed, end-to-end performance [24]. Prior studies surface key constraints (e.g., proof generation time) but stop short of a comprehensive, reproducible analysis of which tuning levers—*batch size/frequency*, *sequencer scheduling*, and *data-availability (DA) compression*—most effectively shift the throughput–latency–cost frontier in real deployments [24, 25]. Despite widespread L2 adoption, we still lack clear guidance on which configuration choices most improve these metrics. By running controlled, reproducible experiments, we aim to identify the true bottlenecks (e.g., proving) and produce practical tuning guidance for the ecosystem [24].

To make the study reproducible and low-friction, we will implement a *simple, educational* proof-of-concept (PoC) ZK-rollup rather than a fully EVM-compatible system (too complex for our scope). The PoC will exercise the target knobs—*batch size/frequency*, *sequencer scheduling*, and *data-availability (DA) compression*—using minimal operations: *token transfers, deposits, withdrawals, and balance updates*. The novel contribution of this work is the creation of an open-source, modular framework specifically designed for controlled, empirical analysis of ZK-Rollup performance trade-offs, which is currently lacking in the ecosystem. While production systems like zkSync and StarkNet are optimized for user adoption and broad feature support, our framework is explicitly designed for scientific benchmarking and reproducibility, prioritizing granular control over production-readiness. We will release the prototype, experiment harness, datasets, and analysis notebooks as *open source* (permissive license, e.g., Apache-2.0) to support future researchers and education; a public repository will host build scripts (Docker), reproducibility configs, and documentation. Prior work reports non-trivial compatibility effort when porting real DeFi code to ZK-Rollups (e.g., adapting Uniswap V2 to zkSync required substantial engineering due to compiler version changes, stricter type handling, and error propagation), hence we avoid full EVM compatibility in this phase [25].

1.2 Background

1.2.1 Blockchain Technology and Scalability Challenges

Blockchain technology underpins decentralized systems, enabling secure, transparent, and tamper-resistant transaction processing through a distributed ledger maintained by a network of nodes [2].

A blockchain consists of a chain of blocks, each containing a list of transactions validated through a consensus mechanism, such as proof-of-work (PoW) or proof-of-stake (PoS). The decentralized nature ensures no single entity controls the network, fostering trust and resilience. However, scalability remains a critical challenge, as blockchains must handle increasing transaction volumes while preserving security and decentralization.

Blockchain scalability refers to the ability of a blockchain network to efficiently process increasing volumes of transactions and user demand without undermining its foundational principles of security and decent ralization [3, 26]. Traditional blockchains like Bitcoin achieve high security and decentralization but suffer from low throughput (e.g., Bitcoin processes approximately 7 TPS). This limitation arises due to constraints in block size, block time, and consensus overhead, which restrict the number of transactions that can be processed in a given time frame.

1.2.2 Scalability of Ethereum

Ethereum, introduced in 2015, extends blockchain functionality beyond simple transactions by supporting smart contracts—self-executing programs that enable complex decentralized applications (dApps). Ethereum's Layer-1 (L1) blockchain, initially based on PoW and later transitioned to PoS via the Ethereum Merge in 2022, processes approximately 15–25 TPS under typical conditions [3, 27]. This throughput is insufficient for global-scale applications, such as decentralized finance (DeFi) or gaming, which demand significantly higher transaction rates. Two primary strategies have emerged to address this limitation: *base-layer (L1) scaling* and *Layer-2 (L2) scaling*.

Modern high-throughput L1s illustrate the design space: Solana targets very high throughput via its Proof-of-History pipeline [28], while Sui's Lutris architecture combines broadcast and consensus for low-latency confirmation [29]. Although these systems can achieve greater raw throughput, they may lack the security budget, liquidity, and ecosystem maturity of Ethereum, raising migration and fragmentation costs [24].

However, L1 changes are complex, requiring consensus upgrades and extensive testing to maintain security. In contrast, L2 scaling leverages off-chain solutions that process transactions outside the main chain while relying on L1 for security and finality. Among L2 solutions, rollups have emerged as Ethereum's preferred scalability mechanism due to their ability to inherit L1 security while significantly increasing throughput [27]. Rollups execute transactions off-chain (on L2), then post succinct data and/or proofs back to L1 to ensure verifiable, secure state transitions. Two main families are prominent:

- **Optimistic Rollups** assume batches are valid by default and rely on fraud-proofs during a challenge window; withdrawals are delayed until the window elapses [30].
- **ZK-Rollups** attach validity proofs that allow immediate verification on L1, enabling faster finality while preserving Ethereum's security model. However, they introduce non-trivial proving and verification costs, and their efficiency and economics remain active areas of study [24].

L1 redesigns can deliver large performance gains but face technical and governance hurdles. L2 rollups scale *today*, yet introduce complexities such as data-availability bandwidth

and proof-generation overhead (especially for ZK variants), and can still experience congestion-driven fee pressure during demand spikes [24, 27]. Both strategies navigate the decentralization–security–scalability trade-space, and ongoing research continues to evaluate their practical efficiency and economic implications.

1.2.3 The Blockchain Trilemma

A foundational concept in blockchain design is the blockchain trilemma (Fig. 1), introduced by Vitalik Buterin [3]. Similar to the CAP theorem in distributed systems [31], the blockchain trilemma highlights the inherent trade-offs among three essential properties of a blockchain system: *security*, *scalability*, and *decentralization* [32]. It asserts that no blockchain can fully optimize all three simultaneously; instead, systems typically achieve two at the expense of the third.

For instance, Bitcoin—the pioneering blockchain—prioritizes security and decentralization to ensure tamper-resistance and censorship-resistance. However, this comes at the cost of scalability, resulting in relatively low transaction throughput [26]. Conversely, attempts to improve scalability often rely on a smaller number of more powerful nodes, which inherently reduces decentralization [33]. Similarly, increasing throughput and confirmation speed to enhance scalability can inadvertently weaken security, introducing new vulnerabilities [33].

The trilemma is therefore not merely descriptive but a fundamental design constraint: no blockchain can maximize all three properties simultaneously. Instead, each system optimizes for two pillars while compromising the third, depending on the requirements of its intended application. Recognizing this trade-off is essential when evaluating scalability solutions, as it shapes what can realistically be considered "success" in decentralized systems.

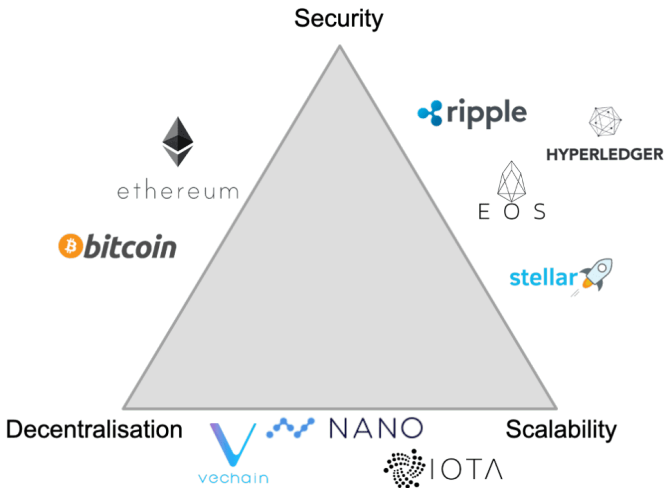

Figure 1: The Blockchain Trilemma

1.2.4 Merkle Trees

A Merkle tree, also known as a hash tree, is a rooted binary tree in cryptography where each leaf node contains the cryptographic hash of a data block, and each internal node holds the hash of its child nodes, culminating in a single Merkle root that serves as a compact commitment

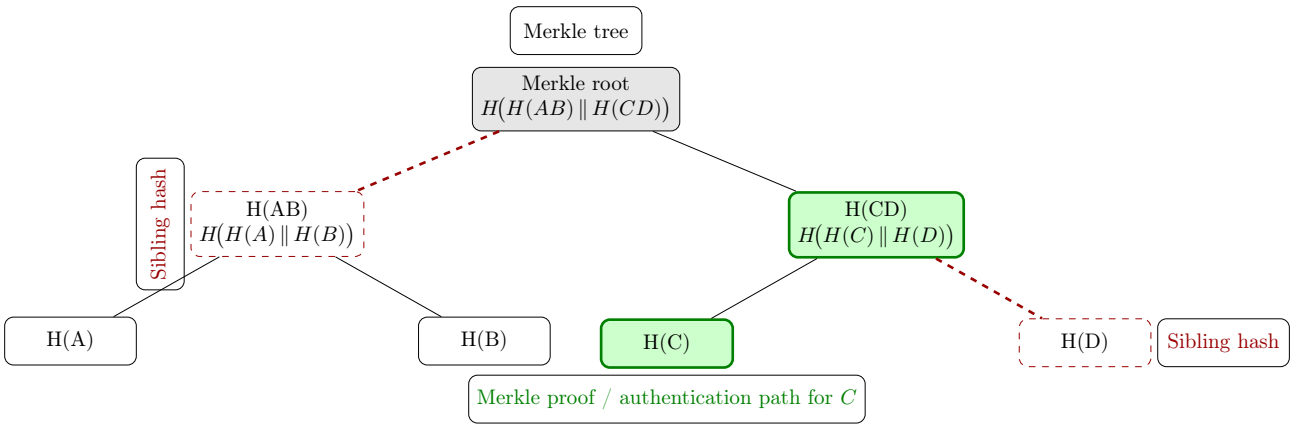

Figure 2: Binary Merkle tree. The inclusion proof (authentication path) for C follows $H(C) \rightarrow H(CD) \rightarrow$ Merkle root (green) and requires the sibling hashes $H(D)$ and $H(AB)$ (dashed red).

to the entire dataset [34]. This structure ensures tamper-evidence, as any modification to a leaf propagates through the hash chain to alter the root, making inconsistencies detectable. Membership verification is efficient, requiring only an $O(\log n)$ Merkle proof consisting of sibling hashes along the path from the leaf to the root, which allows recomputation and comparison without accessing the full tree [35]. Common implementations employ collision-resistant hash functions like SHA-256, with variants including binary (two children per node) or k-ary trees for optimized depth in large datasets [36].

Merkle trees can be used in distributed systems for efficient data verification and synchronization.

1.2.5 Zero-Knowledge Proofs

Zero-Knowledge Proofs (ZKPs) are a foundational cryptographic primitive that enables one party (the prover) to demonstrate to another party (the verifier) that a specific assertion is true without revealing any additional information beyond the truth of the statement itself. The prover possesses some secret information—such as a private key or the solution to a computational problem—and can convince the verifier of a claim (for example, knowledge of the secret) by providing a cryptographic proof that reveals nothing else about the secret [37]. This concept was first introduced in 1985 by Shafi Goldwasser, Silvio Micali, and Charles Rackoff in their seminal paper "The Knowledge Complexity of Interactive Proof-Systems," which laid the groundwork for a new paradigm in cryptographic protocols and security research [37].

ZKPs can be categorized into interactive and non-interactive variants.

- **Interactive ZKPs:** as originally proposed, involve multiple rounds of communication between the prover and verifier, where the verifier issues challenges and the prover responds accordingly to build confidence in the proof's validity. A small detail is that this interactivity ensures soundness through randomness in challenges, but it requires online participation from both parties, which can be inefficient for certain applications [37].

- **Non-interactive ZKPs (NIZKs):** on the other hand, eliminate the need for back-and-forth interaction by transforming the protocol into a single-message proof, often using techniques like the Fiat-Shamir heuristic, which replaces verifier challenges with hash-based pseudorandom values derived from the prover's initial commitment [38]. This makes NIZKs more practical and suitable for blockchain scenarios, where efficiency, offline verification, and scalability are crucial, though they may rely on additional assumptions like the random oracle

model [38]. Specialized non-interactive forms include zk-SNARKs, which are highly succinct and efficient, making them the most suitable for blockchain applications due to small proof sizes and fast verification times, and zk-STARKs, which offer transparency without trusted setups and post-quantum security but with larger proofs [39–42].

In the context of blockchain technology, ZKPs allow for the verification of transaction validity or state transitions without exposing sensitive data. This capability makes them particularly useful for enhancing privacy and scalability in decentralized systems. A robust ZKP protocol must satisfy three core properties [43]:

- **Completeness:** If the statement is true and both parties follow the protocol, the honest verifier will accept the prover's proof.
- **Soundness:** If the statement is false, no dishonest prover can convince an honest verifier that it is true, except with negligible probability.
- **Zero-Knowledge:** The verifier learns nothing from the proof apart from the fact that the statement is true (no additional knowledge is leaked).

Collectively, these properties ensure that ZKPs can be applied securely in trust-minimized, decentralized environments, where verifiability is paramount. In other words, even on a public blockchain, one can prove a transaction or computation is valid without revealing the underlying private data, preserving confidentiality while still guaranteeing correctness [44, 45].

1.2.6 zk-SNARKs

Building on the principles of ZKPs, zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge) represent a specialized subset of non-interactive zero-knowledge proofs optimized for efficiency in blockchain applications; they are considered the most suitable for blockchain due to their ability to handle complex computations with minimal overhead in resource-constrained environments [39, 46]. As non-interactive ZKPs, zk-SNARKs allow the prover to generate a single, compact proof that the verifier can check without ongoing communication, making them highly suitable for distributed systems [38]. The defining features of zk-SNARKs are their succinctness (proofs are very small and quick to verify) and non-interactivity (the proof can be verified without any back-and-forth communication between prover and verifier). These properties make zk-SNARKs ideal for resource-constrained environments like public blockchains, supporting scalable and privacy-preserving operations without compromising security [39, 44]. In practice, zk-SNARK-based systems can verify complex computations with minimal overhead, which is why they are used in applications ranging from privacy coins (e.g., Zcash) to Layer-2 rollups for scaling Ethereum [47].

A prominent example of a zk-SNARK system is the Groth16 protocol, proposed by Jens Groth in 2016. Groth16 operates through three main steps [39, 43]:

- **Key Generation:** A trusted setup phase generates a public proving key and verification key from some secret parameters.
- **Proof Generation:** Using the proving key, the prover computes a compact proof of the statement's truth (for a given private witness and public inputs).
- **Verification:** Using the verification key, the verifier checks the proof's validity. If the proof is valid, the statement is accepted as true.

Groth16 is highly efficient—its proofs can be as small as three group elements, and verifying a proof requires only a single pairing check. This efficiency has made Groth16 a cornerstone of many ZK-rollups and privacy-focused systems like Zcash [39]. However, Groth16 requires a trusted setup; if the setup is compromised (i.e., if the secret trapdoor from the key generation were leaked or misused), the soundness of the system could be undermined. This limitation has prompted ongoing research into setup-free or universal setup alternatives (such as Sonic, PLONK, and zk-STARKs) that remove or downplay the need for a trusted setup [42, 48–51].

1.2.7 zk-STARKs

zk-STARKs (Zero-Knowledge Scalable Transparent ARguments of Knowledge) are another advanced form of non-interactive ZKPs, distinguished by their transparency and scalability. Unlike many zk-SNARKs, zk-STARKs do not require a trusted setup, relying instead on public randomness and collision-resistant hash functions for security. This makes them "transparent" and resistant to certain attacks associated with secret parameters [49]. A small detail is that zk-STARKs use algebraic intermediate representations (AIR) and low-degree testing via the FRI protocol to achieve post-quantum security, though this comes at the cost of larger proof sizes compared to zk-SNARKs [52]. As non-interactive proofs, they enable efficient verification of large-scale computations without interaction, making them suitable for high-throughput blockchain scaling solutions [49]. zk-STARKs are particularly valuable in environments demanding quantum resistance and full transparency, such as advanced Layer-2 protocols [52].

1.2.8 Data Availability

Data Availability (DA) in distributed systems ensures that all necessary data for verifying state transitions is accessible, preventing data withholding attacks where malicious entities might conceal information [53]. DA mechanisms can be categorized into on-chain and off-chain approaches: on-chain DA publishes transaction data or state diffs directly to the base layer (e.g., via calldata or EIP-4844 blobs for cost efficiency), while off-chain DA relies on external storage solutions with on-chain commitments (e.g., data availability committees or layers like Celestia using erasure coding and sampling) [25, 54]. This ensures data integrity and retrievability, with trade-offs in cost, security, and scalability. Data availability can be used in distributed systems for efficient data verification and synchronization.

1.3 Problem Statement

Ethereum, a leading public blockchain, faces significant scalability limitations that hinder its ability to support high-volume decentralized applications (dApps) such as decentralized finance (DeFi), non-fungible tokens (NFTs), and gaming [3]. Under typical conditions, Ethereum's Layer-1 (L1) processes approximately 15–25 transactions per second (TPS), far below the throughput needed for global-scale adoption [32, 55]. This low capacity leads to congestion, volatile fees, and delayed finality, reflecting the blockchain trilemma where security and decentralization are prioritized over base-layer scalability [56].

Zero-knowledge rollups (ZK-rollups) have emerged as a promising Layer-2 (L2) approach [57]. By executing transactions off-chain and submitting succinct validity proofs (e.g., zk-SNARKs or zk-STARKs) to L1, ZK-rollups can, in principle, deliver substantially higher

throughput while inheriting L1 security. However, real deployments still face practical bottlenecks: [24, 58]

1. Computationally intensive proof generation that slows finality
2. Sequencer scheduling inefficiencies under congestion
3. High data-availability (DA) costs—even with EIP-4844—that raise fees

This project addresses these gaps by implementing and empirically evaluating a modular ZK-rollup prototype on an Ethereum testnet (e.g., Sepolia). We will benchmark baseline metrics against L1 and explore targeted optimizations—dynamic batch sizing, improved sequencer policies, and DA compression strategies (including EIP-4844 blobs)—to enhance throughput, reduce latency, and minimize costs without compromising security [58]. Through controlled experiments and new datasets, we aim to produce actionable, reproducible evidence that bridges theoretical scalability claims and practical outcomes.

1.4 Research Questions

To guide this investigation, the project will answer:

1. **RQ1:** How can we implement a *modular* ZK–Rollup with runtime-swappable modules for **batching**, **sequencing**, **state execution**, **proving**, **commitment/submitter**, and **data availability (DA)**?
2. **RQ2:** How do **batch size & frequency**, **sequencing policies**, and **DA modes** (L1 calldata vs. EIP-4844 blobs) affect end-to-end performance (**TPS**, **latency**, **gas/tx**, **proof time**)?
3. **RQ3:** To what extent can **benchmark-driven tuning** improve the scalability of a modular ZK–Rollup?

These questions aim to yield **empirical, reproducible** evidence to guide ZK–Rollup design and optimization.

1.5 Research Objectives

We address the RQs via development, controlled experimentation, and analysis, emphasizing modularity and reproducibility:

1. **Design & implement** a modular ZK–Rollup prototype with swappable components for **batching**, **sequencing**, and **DA** (addresses RQ1).
2. **Benchmark systematically** by varying **batch size/frequency**, **sequencing policies** (e.g., FIFO vs. fee-priority), and **DA modes** (calldata vs. EIP-4844 blobs) to measure **TPS**, **latency**, **gas/tx**, and **proof time** (addresses RQ2).
3. **Analyze & tune** to quantify gains from:
   - **Batch** size/frequency rules (optimize proof overhead and bundling),
   - **Sequencer** scheduling (including decentralized/shared variants),
   - **DA** posting/compression (e.g., leveraging EIP-4844 blobs).

   Produce actionable configuration guidance and validate improvements (addresses RQ3).

1.6 Research Outcomes

Upon completion, this project will yield several key outcomes that advance both academic understanding and practical deployment of ZK-Rollups in Ethereum ecosystems. These outcomes are designed to be open and extensible, promoting further research and real-world applications:

1. A fully functional, modular ZK-Rollup prototype deployed on the Ethereum testnet, complete with configurable components for batching, sequencing, and DA handling (e.g., integrated with EIP-4844 for blob-based storage).

2. Empirical insights from optimization experiments, including quantitative trade-off analyses (e.g., batch size increases yielding 20–50% throughput gains but 10–30% higher latency) and recommendations for sequencer decentralization to mitigate single-point failures.

3. Open-source artifacts, including the prototype codebase (e.g., on GitHub under Apache-2.0 license), experiment harnesses (Dockerized for reproducibility), raw datasets, and Jupyter notebooks for analysis, enabling community validation and extensions (e.g., integration with DeFi protocols).

These outcomes will not only validate optimization strategies but also contribute to sustainable blockchain scaling, potentially reducing L2 fees by orders of magnitude and supporting applications in education, finance, and beyond.

2 Literature Review

2.1 Executive Summary

Ethereum's limited throughput has driven the rise of Layer-2 solutions [1,59], with ZK-Rollups emerging as the most promising due to their use of validity proofs for fast, secure finality [58, 60]. Existing research highlights key challenges around sequencer centralization [53], the high and evolving costs of data availability [61], and computational bottlenecks in proof generation [25, 58]. Trade-offs between batch size, latency, and cost further complicate rollup design, while simple TPS metrics fail to capture real scalability limits. Although systems like zkSync [62] and StarkNet [63] demonstrate production viability, their complexity prevents controlled experimentation [53,61,64]. Current gaps include the lack of end-to-end empirical evaluations and fine-grained studies of batching, scheduling, and data availability strategies. This project addresses these gaps through a minimal, modular ZK-Rollup prototype, enabling reproducible benchmarking and generating actionable insights into the scalability trade-offs of rollup design.

2.2 The Ethereum Scalability Challenge and the Rise of Layer 2

The proliferation of decentralized applications (dApps) has exposed a fundamental limitation of legacy Layer-1 (L1) blockchains like Ethereum [27]. Designed for security and decentralization, Ethereum's architecture currently constrains its transaction throughput to a range of 12 to 17 transactions per second (TPS) [59], leading to network congestion, high transaction fees, and a poor user experience [59]. This inherent scalability bottleneck has become a significant barrier to mainstream adoption, compelling the blockchain ecosystem to seek alternative solutions [59].

To address these challenges without altering the foundational L1 design, a new class of "off-chain" scaling solutions, known as Layer-2 (L2) protocols, has emerged. These systems process transactions outside the main blockchain, leveraging the L1 for security and data integrity. Among the various L2 alternatives, such as state channels and sidechains, rollups have distinguished themselves as the most promising and widely adopted technology [27, 59, 61]. They operate by executing transactions and state computations off-chain, then bundling the data into a single, compressed transaction that is committed back to the L1 [64].

2.3 Zero-Knowledge Rollups: Current State of the Art

The rollup landscape is defined by two primary technologies: optimistic rollups and Zero-Knowledge Rollups (ZK-Rollups). While optimistic rollups rely on a fraud proof system that assumes transactions are valid by default and requires a challenge period to identify and penalize fraudulent state transitions, ZK-Rollups leverage cryptographic validity proofs to verify the correctness of state transitions before they are committed to the L1 [53]. This crucial distinction allows ZK-Rollups to offer fast transaction finality, a key advantage that has led them to be regarded as a leading solution for L2 scalability [60]. A typical ZK-Rollup system includes four main off-chain components:

- **Sequencer** responsible for collecting, ordering, and queuing pending user transactions.

- **Executor:** which processes transactions and constructs transaction blocks to update the system state, often represented in Merkle trees [65].
- **Submitter:** which broadcasts transactional data, such as state differences and Merkle tree roots, to the L1 after execution.
- **Prover:** which generates cryptographic validity proofs to substantiate the computational integrity of state transitions [53].

2.3.1 Sequencer Scheduling Policies

The sequencing policy determines how transactions are ordered, with implications for fairness, latency, and cost efficiency. Literature identifies several approaches:

- **First-Come-First-Serve (FCFS):** simple and fair but vulnerable to latency racing [53].
- **Maximal Value:** prioritizes fees and sequencer profit, but may delay inclusion [25].
- **Time-Boost:** allows users to pay for faster confirmation, introducing tiered access [53].
- **Fair BFT Ordering:** emphasizes time-based fairness, reducing collusion and latency-racing risks [53].

Each policy presents trade-offs between fairness, performance, and economic incentives, highlighting the difficulty of balancing user experience with system efficiency [53].

Sequencer Efficiency and Bottlenecks

Sequencer performance strongly influences throughput, latency, and finality.

- **Batching:** Larger batches reduce average transaction costs but increase latency, while smaller batches improve responsiveness at the expense of efficiency [27].zkSync Era, for instance, achieves soft finality in $<$2.5s for over 50% of transactions but requires 10–20 minutes for L1 settlement [25].
- **Computational Bottlenecks:** Proof generation is sensitive to batching [58], with Merkle tree computation adding up to 2.44s per batch [25]. Design Differences: zkSync leverages data compression to support large batches on modest hardware [58], whereas Polygon zkEVM achieves more predictable proving times but requires expensive infrastructure [58].
- **Scalability:** Benchmarks demonstrate 71 TPS for zkSync swap transactions (vs. 15 TPS on Ethereum L1) [25], yet stress testing reveals instability under high load, underscoring the fragility of centralized sequencer designs [25].

2.3.2 Future Directions

Ongoing research emphasizes:

- Adaptive scheduling mechanisms to optimize batching under bursty workloads [58].
- Opcode-aware batching strategies that align transaction grouping with prover capacity [58].
- Metering and incentive models to mitigate DoS risks and align sequencer behavior with network health [60].
- Sequencers-as-provers architectures (e.g., Starknet) to improve efficiency, though at the cost of heightened collusion risk [53].

2.3.3 Data Availability and Its Evolving Cost Dynamics

Data Availability (DA) is a foundational aspect of ZK-Rollups, ensuring that transaction data required for verifying state transitions is always accessible on Layer 1 (L1). Without DA,

sequencers could withhold data, preventing users from validating the rollup state or recovering funds [61].

2.3.3.1 On-Chain DA

ZK-Rollups traditionally rely on on-chain data posting, securing transactions through Ethereum's consensus but at high cost.

- Calldata: Early ZK-Rollups such as Loopring and Immutable X compressed state diffs into calldata [25]. This guaranteed security but was limited by high gas costs (16 gas per non-zero byte) and block size restrictions.
- EIP-4844 Blobs: Introduced in March 2024, Proto-Danksharding added blob-carrying transactions, allowing 128 KB per blob and up to 2 MB per block [25]. Rollups like zkSync Era, Linea, Starknet, and Scroll adopted blobs to cut DA costs [25]. However, blobs are sold in fixed sizes, meaning small batches may be more efficient with calldata [25].
- Compression: ZK-Rollups increasingly post state differences instead of full transaction data to optimize DA. zkSync Era's compression outperforms Polygon zkEVM, reducing per-transaction DA costs [25].

2.3.3.2 Off-Chain DA

Some ZK-Rollups explore validium mode, where data is kept off-chain for lower costs but verified via external mechanisms.

- DACs (Data Availability Committees): Trusted committees sign and store data off-chain, as seen in Immutable X [25].
- DALs (Data Availability Layers): Permissionless networks like Celestia and Avail use erasure coding and data availability sampling to guarantee scalability and reduce costs [66].

2.3.3.3 Evolving Cost Dynamics

DA historically represented 80% of ZK-Rollup transaction costs [67]. With EIP-4844 blobs, DA costs have dropped significantly, making proof generation the dominant cost. Key trade-offs remain:

- Batch size vs. finality: Larger batches lower per-transaction DA costs but slow finality [58].
- Sequencer optimization: Sequencers reduce DA overhead by batching related transactions or grouping workloads by prover limits [58].
- Blob pricing challenges: Fixed blob sizes remain inefficient for small rollups, and the long-term blob market is uncertain [25].

2.3.4 Proving Bottlenecks and Nuanced Performance Metrics

The prover, responsible for generating validity proofs, is a major computational bottleneck. While simple token transfers have been shown to achieve theoretical throughput above 14,000 TPS, real-world workloads like DeFi swaps achieve only 71 TPS. This discrepancy highlights the inadequacy of simple TPS metrics: true scalability depends on transaction complexity [25]. Proof generation costs also increase with batch size, and Merkle tree computation has been identified as a dominant delay (up to 2.44s per batch). These findings emphasize the need to study proving bottlenecks more carefully [25].

2.3.5 Finality, Cost, and Efficiency Trade-offs

ZK-Rollup design involves a trade-off between throughput, latency, and cost [53]. Larger batches are cheaper per transaction due to amortized fixed costs (e.g., L1 posting and proof verification), but they increase latency since sequencers must wait to fill batches. Another trade-off is between soft finality (fast L2 confirmation, $<$2.5s) and hard finality (L1 confirmation, 10–20 minutes) [25]. While users prefer fast confirmations, security ultimately depends on slower L1 settlement. Literature suggests that over 50% of transactions achieve soft finality within 2.5s, but finality guarantees vary widely [25].

2.4 Gaps in the Literature

Despite progress, current research exhibits key gaps. Our review of the literature reveals a clear, overarching gap: while the core components of ZK-Rollups are well-understood in isolation, there is a lack of holistic, end-to-end empirical analysis of how their parameters interact to affect real-world performance.

- Limited end-to-end empirical evaluations of ZK-Rollups: Controlled, end-to-end benchmarks of ZK-Rollups are sparse, particularly for real-world workloads.
- Lack of controlled studies on how batch size, scheduling, and DA strategies interact to affect throughput, latency, and cost.
- Insufficient fine-grained cost breakdowns in real-world settings.
- Little analysis of the interaction between sequencer, prover, and DA within fee mechanism design.

This project aims to fill these gaps by building a minimal, modular ZK-Rollup prototype supporting basic token transfers. Unlike production-ready systems such as zkSync, Polygon zkEVM, and StarkNet, which are too complex for fine-grained experimentation, the prototype will allow controlled benchmarking of sequencing policies, batching strategies, and DA compression. This approach enables reproducible empirical analysis that validates or challenges theoretical claims while producing actionable insights for the ZK-Rollup ecosystem.

3 Proposed Method

The core of this project is the design, implementation, and rigorous benchmarking of a modular ZK-Rollup framework. This framework will be deployed on an Ethereum testnet to ensure that our findings are grounded in a realistic network environment. The initial implementation will support a focused set of operations—namely, simple token transfers, deposits, and withdrawals—which serve as the foundation for our performance analysis. A key feature of the framework is its configurability, which will allow us to systematically vary experimental parameters such as batch size, sequencer scheduling policies, and data availability modes. Furthermore, the architecture will be intentionally designed to be extendable, thereby allowing for future evolution and the integration of more complex functionalities.

3.1 Prototype Design and Implementation

3.1.1 System Architecture

To ensure a clear separation of concerns and to facilitate modular experimentation, the prototype will be structured into a five-layer architecture. Each layer will have a distinct responsibility, from transaction submission at the top to data availability at the bottom. This design choice is critical for isolating variables during benchmarking and for allowing future extensions to the framework.

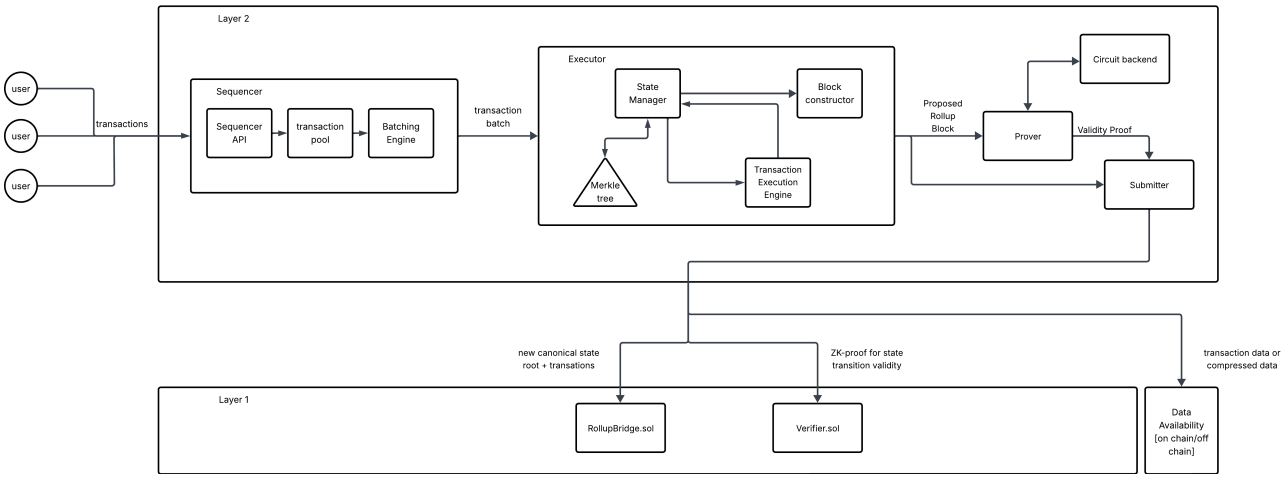

Figure 3: High level System Architecture

The components of the prototype are as follows:

- **User Layer:** The user layer is the entry point for end-users to interact with the system.
  - Wallets / Clients: End-users initiate deposits, transfers, and withdrawals by signing transactions.
  - Synthetic Workload Generator: Provides benchmark workloads for performance evaluation.
- **Off-Chain Components:** These components handle the core logic of the rollup, including transaction processing, state management, and proof generation.
  - **Sequencer:** The sequencer is responsible for collecting and ordering transactions.
    - Transaction Pool: Collects and stores pending user transactions.
    - Batching Engine: Groups transactions into batches based on configurable policies (e.g., batch size, timeout, or priority).

Table 2: Configurability & Data-Availability Policy Matrix — zkSync Era vs Polygon zkEVM vs Our Modular PoC

Parameter	zkSync Era	Polygon zkEVM	Our Modular PoC
Batch size	○ Partial — sequencer-set	○ Partial — sequencer-set	✓ Config flag
Sequencing policy	✗ Operator-controlled; no priority fees	✗ Trusted sequencer; `sequenceBatches()`	✓ FIFO / fee-priority / custom
Commitment policy	✗ Protocol rules	✗ Protocol rules	✓ Tunable interval / thresholds
Prover configuration	✗ Internal state-diff / compression	✗ Fixed protocol stack	✓ Pluggable; parallelism knobs
State update policy	✗ Internal state-diff / compression	✗ Protocol-defined	✓ Tree / commit granularity
DA config / policy	✗ On-chain DA; calldata & EIP-4844 blobs	✗ On-chain DA; EIP-4844 enabled	✓ Switchable: on-chain & off-chain compression toggles

Legend: ✓ = user-configurable, ○ = partial/limited by sequencer, ✗ = protocol-/operator-controlled.

    - Scheduler (optional): Supports custom policies for sequencing order (FIFO, priority-based).
  - **Executor:** The executor processes the transactions and updates the off-chain state.
    - State Manager: Maintains the off-chain state (i.e., account balances and nonces) using a Sparse Merkle Tree (SMT). We plan to adapt an existing JavaScript library such as `js-merkle` or the implementation used by Polygon's zkEVM for this purpose.
    - State Transition Engine: Applies transactions to update the state and compute new state roots.
    - Block Constructor: Forms transaction blocks with corresponding state updates.
  - **Prover:** The prover generates cryptographic proofs to ensure the correctness of state transitions.
    - Circuit Backend: zkSNARK/STARK circuits verify state transitions (OldRoot → NewRoot) [68].
    - Default Backend: Circom + Groth16/Plonk for rapid prototyping [69–71].
    - Optional Backends: Plonky2 or Halo2 for performance comparisons [71, 72].
    - Proof Generator: Produces validity proofs ensuring correctness of state transitions.
  - **Submitter:** The submitter packages and sends the rollup data to the Ethereum mainnet.
    - Batch Packager: Packages transactions, state roots, and validity proofs.
    - On-chain Broadcaster: Submits canonical state roots, ZK proofs, and DA commitments to Ethereum.
    - Data Availability Connector: Chooses DA mode (calldata, compression, or external DA).
- **On-Chain Components** These are the smart contracts deployed on the Ethereum mainnet that serve as the anchor for the rollup.
  - **RollupBridge.sol**
    - Stores canonical state roots.
    - Manages deposits and withdrawals.

    - Emits batch events for client synchronization.
  - **Verifier.sol**
    - Verifies zkSNARK/STARK proofs submitted by the off-chain prover.
    - Enforces correctness of all rollup state transitions.
- **Data Availability Layer** This layer specifies how the transaction data is made available for anyone to reconstruct the rollup's state.
  - Mode A: Store full transaction data in Ethereum calldata.
  - Mode B: Store compressed transaction data (delta encoding, bit-packing).
  - Mode C: Store data off-chain (IPFS [73] or Celestia [66] like DA layers) with on-chain pointers.

3.1.2 Prototype Implementation Steps

The development of the prototype will follow a layered approach, starting from the user-facing layer, moving through the off-chain execution pipeline, and finally integrating with on-chain contracts and the data availability layer. This phased structure ensures iterative validation and controlled experimentation.

The key implementation steps are as follows:

- **User Layer Development:** The first step will be to integrate with existing browser-based wallets (e.g., MetaMask) to allow users to initiate deposits, transfers, and withdrawals. In parallel, a synthetic workload generator will be built to simulate benchmark workloads for stress testing and performance evaluation.
- **Off-chain Sequencer and Executor:** A minimal sequencer will be implemented in Node.js/TypeScript to handle the transaction pool, batching engine, and (optionally) custom scheduling policies (FIFO, priority-based). The executor will be developed alongside, featuring a Sparse Merkle Tree (SMT)-based state manager, a state transition engine for applying transactions, and a block constructor to produce ordered blocks with updated state roots.
- **Prover Integration:** Circuits will be designed to verify token balance updates and state transitions (OldRoot $\rightarrow$ NewRoot) [68]. The initial backend will use Circom with Groth16/Plonk for rapid prototyping [69–71], while optional backends such as Plonky2 and Halo2 [71, 72] will be explored for performance comparisons. Proof generation will then be integrated with the sequencer and executor pipeline.
- **Submitter and On-chain Contracts:** A submitter component will be developed to package transactions, state roots, and proofs, and broadcast them to Ethereum. On-chain contracts, namely `RollupBridge.sol` and `Verifier.sol`, will serve as the canonical anchors by storing state roots, managing deposits/withdrawals, emitting synchronization events, and verifying zkSNARK/STARK proofs.
- **Data Availability Layer:** Different DA strategies will be integrated and tested: storing full transaction data in Ethereum calldata (Mode A), compressed calldata (Mode B), and off-chain storage solutions such as IPFS [73] or Celestia [66] with on-chain commitments (Mode C).
- **Benchmarking and Experimental Harness:** Once the pipeline is functional, benchmarking scripts will be developed to drive synthetic workloads (modeled using a Poisson process) and collect metrics such as throughput (tx/sec), latency, gas cost, storage footprint, and DA efficiency. This harness will allow systematic experimentation across batch sizes, timeouts, scheduling policies, proof backends, and DA modes.
- **Modular and Extensible Framework:** Throughout implementation, the system will be designed as a modular framework, drawing inspiration from zkSync and Polygon zkEVM, while avoiding direct code reuse. This ensures transparency and control in experimentation,

while enabling future extensions to richer transaction types (ERC20, ERC721 [74,75]), multi-sequencer architectures, and zkEVM circuits for general-purpose contract execution.

3.2 Experimental Design for Optimization Strategies

3.2.1 Objectives

Our goal is to quantify how design choices across the *sequencer, executor, prover, submitter, and data-availability (DA)* layers shape end-to-end performance of a modular ZK–Rollup. We isolate parameters and run controlled sweeps to expose trade-offs in throughput, latency, and cost (with reproducible instrumentation).

- **Sequencer layer (batching & scheduling).** Evaluate the effect of *batch size* and *batch frequency* (time-/size-triggered) and *scheduling policies* (e.g., FIFO vs. fee-priority) on TPS, confirmation-latency percentiles (P50/P95), tail behavior under contention, and fairness across flows.
- **Executor / state-update layer.** Assess state-update efficiency under different *state-tree / commitment* options and *block construction* strategies (e.g., miniblocks vs. macro-batches), measuring execution time per block, commit cadence, and bytes committed.
- **Data availability strategies.** Compare Mode A: *raw calldata*, Mode B: *compressed/state-diff calldata*, and Mode C: *off-chain DA with on-chain commitments* (e.g., DAC/DAL) in terms of DA bytes, gas/tx, and end-to-end latency; include external-DA options (e.g., content-addressed storage, modular DA layers) [66,73].
- **Proof-system backends.** Contrast `Groth16`, `Plonky2`, and `Halo2`—including recursion when applicable—for *prover wall-clock time*, *parallelism scaling*, *peak memory*, and *L1 verification cost (gas)* [69–72].
- **Multi-objective trade-off mapping.** For each configuration, record **TPS**, **end-to-end and confirmation latency**, **gas/tx** (broken down by execution, proof verify, DA), **proof generation time**, and **DA bytes**. Fit response surfaces over *batch size* × *scheduler* × *DA mode* and report Pareto frontiers to guide scalable rollup deployments.

3.2.2 Independent Variables (Experimental Factors)

Factor	Description
Batch size	Number of transactions per batch
Batch frequency / time-out	Time interval to flush transactions into a batch
Sequencer scheduling policy	Determines transaction ordering in the batch
Data Availability mode	How transaction data is made available for verification
Proof system	Type of zk-proof used for batch verification

3.2.3 Dependent Variables (Metrics to Measure)

Metric	Description
Throughput	Number of transactions processed per second (tx/sec)
Latency	Time from transaction submission → final verification on-chain
Gas cost	Gas used per batch and per transaction
Proof generation time	Time taken to compute zk-proof for a batch
Verification time	Time for on-chain verifier to validate the proof
Data availability effect	Comparison of gas and storage under different DA modes

3.3 Experimental Procedure

Our experimental procedure is designed to be systematic and reproducible, ensuring that our findings are both reliable and verifiable. The procedure is divided into four main phases: setup, controlled experiments, data collection, and analysis.

3.3.1 Setup

The setup phase involves preparing the environment for our experiments:

- **Deployment:** We will deploy the prototype ZK-Rollup on two environments: a public Ethereum testnet (e.g., Sepolia) for realistic, public-facing benchmarks, and a local testnet (e.g., Hardhat) for controlled, reproducible experiments that mitigate the "observer effect" of public network congestion.
- **Baseline Establishment:** To provide a clear point of comparison, we will establish a baseline by deploying a simple L1 smart contract that performs the same operations as the ZK-Rollup. This will allow for a direct comparison of gas costs and throughput between our L2 solution and the L1.
- **Module Initialization:** We will initialize all the core modules of our prototype, including the Sequencer, Prover, RollupBridge, and Verifier.
- **Instrumentation:** We will configure our benchmarking scripts to log all relevant data, including timestamps, gas consumption, and storage usage, for each experiment.

3.3.2 Controlled Experiments

In this phase, we will conduct a series of controlled experiments, manipulating one independent variable at a time while keeping others constant:

- We will vary the batch size (e.g., testing values from 10 to 1,000 transactions) and measure its impact on throughput, latency, and gas costs.
- We will vary the batch frequency and timeout to observe the trade-off between latency and throughput.
- We will apply different sequencer scheduling policies and measure their effect on fairness (e.g., average and 95th percentile latency) and overall throughput.
- We will switch between different data availability modes to compare their on-chain gas costs, storage requirements, and verification latencies.
- Where possible, we will swap out the underlying proof systems to measure differences in proof generation and verification times.

3.3.3 Data Collection

To ensure the reliability of our results, we will follow a rigorous data collection process:

- We will capture all defined metrics for each transaction batch.
- Each experiment will be repeated multiple times to ensure statistical significance and to account for any variability in the test environment.
- All results will be stored in a structured format (e.g., CSV or JSON) to facilitate subsequent analysis.

3.3.4 Analysis and Validation

The final phase of our experimental procedure involves analyzing the collected data and validating our findings:

- **Analysis:** We will plot a series of trade-off curves, such as throughput vs. latency, gas cost vs. batch size, and proof generation time vs. batch size. These visualizations will help us to identify the optimal configurations for different goals (e.g., minimum latency vs. maximum throughput) and to evaluate the impact of DA and proof systems on the overall efficiency of the rollup.
- **Validation:** We will compare our observed performance with theoretical predictions from the ZK-Rollup literature. This will allow us to highlight any discrepancies between theory and practice and to identify potential bottlenecks in our prototype's design.

3.3.5 Threats to Validity

To ensure a rigorous evaluation, we acknowledge potential threats to the validity of our findings:

- **Internal Validity:** Bugs in our synthetic workload generator or benchmarking scripts could introduce measurement errors or unrealistic transaction patterns, affecting the reliability of our results. We will mitigate this through careful code review and by testing the generator against known patterns.
- **External Validity:** Our findings on the Sepolia testnet, which has different gas price dynamics and network congestion patterns, may not generalize perfectly to the Ethereum mainnet. Furthermore, the performance of our prototype, which is not a production-grade system, may differ from that of fully optimized rollups. We will address this by clearly documenting our experimental environment and highlighting these limitations in our analysis.

4 Research Timeline

This section outlines the timeline for the project, including the various phases, key milestones, and the overall schedule depicted in the Gantt chart. The timeline is designed to ensure the project progresses smoothly and all deliverables are met within the specified timeframe.

4.1 Project Phases

The key project phases are structured to guide the project from initiation to completion. Each phase has specific objectives and tasks that contribute to the successful delivery of the project. The phases are:

1. Project Initiation and Research: Identify the research problem, define project scope and objectives and conduct the Literature Review.
2. System Architecture Design: Define system architecture and plan experimental parameters.
3. Prototype Implementation: Developing the modular zk-rollup framework, testing and deploying on an Ethereum testnet.
4. Conduct Experiments, Benchmarking and Optimization: Establish benchmarking metrics, conduct experiments and optimization.
5. Data Analysis: Perform statistical analysis and validate findings.
6. Documentation and Presentation: Prepare the final documentation, publications, and present the project results.

4.2 Milestones

To monitor progress and ensure timely completion, the project includes several key milestones. These milestones mark significant points in the project and serve as checkpoints to review progress. The milestones are:

1. Literature Review Completion: Completing the review of relevant literature and research.
2. Project Proposal Submission: Submitting the detailed project proposal.
3. Prototype Implementation: Deploy functional zk-rollup prototype on an Ethereum testnet.
4. Complete Experiments: Complete optimization experiments, analysis and optimizations.
5. Research Paper Submission: Submitting the research paper detailing the project outcomes.
6. Final Report Submission: Submitting the comprehensive final report of the project.

4.3 Gantt Chart

The following Gantt chart conveys the estimated timeline, project deliverables, and the planned stages of the project. It provides a visual representation of the project schedule and helps in tracking the progress of each phase and milestone.

Task Name	Start	End	Duration	Progress %
Project Initiation	2025-07-14	2025-09-14	63 days	100
Identify the research problem	2025-07-14	2025-08-09	27 days	100
Define project scope and objectives	2025-07-14	2025-08-16	34 days	100
Study the domain area	2025-07-14	2025-09-14	63 days	100
Literature review	2025-07-14	2025-09-14	63 days	100
Project proposal	2025-08-10	2025-09-14	36 days	100
System Architecture Design	2025-08-31	2025-09-20	21 days	67
Define system architecture	2025-08-31	2025-09-13	14 days	100
Setup development environment	2025-09-07	2025-09-20	14 days	35
Prototype Implementation	2025-09-21	2025-12-19	90 days	0
Implement Ethereum contracts	2025-09-21	2025-10-11	21 days	0
Implement the sequencer	2025-09-21	2025-10-18	28 days	0
Implement the executor	2025-10-05	2025-11-01	28 days	0
Implement the prover	2025-10-19	2025-11-22	35 days	0
Implement the submitter and DA modes	2025-11-09	2025-12-06	28 days	0
Prototype integration and testing	2025-09-28	2025-12-06	70 days	0
Deploy prototype on an Ethereum testnet	2025-11-30	2025-12-19	20 days	0
Conduct Experiments, Benchmarking and Opti…	2025-12-13	2026-02-01	51 days	0
Establish benchmarking metrics	2025-12-13	2025-12-19	7 days	0
Develop benchmarking scripts	2025-12-13	2025-12-26	14 days	0
Conduct controlled experiments	2025-12-20	2026-02-01	44 days	0
Benchmarking data collection	2025-12-20	2026-02-01	44 days	0
Optimization	2025-12-20	2026-02-01	44 days	0
Data Analysis	2026-01-26	2026-02-08	14 days	0
Perform statistical analysis	2026-01-26	2026-02-08	14 days	0
Validate findings	2026-02-02	2026-02-08	7 days	0
Documentation and Presentation	2026-02-09	2026-05-08	89 days	0
Documentation of the prototype	2026-02-09	2026-02-15	7 days	0
Prepare research paper and publications	2026-02-16	2026-04-26	70 days	0
Prepare final report	2026-02-23	2026-04-26	63 days	0
Final presentation	2026-04-27	2026-05-08	12 days	0

Figure 4: Gantt Chart for the project timeline.

4.4 Workload Calculation

The total estimated workload for the whole project is 2,240 hours. The following table provides a detailed breakdown of the time that will be spent on each task.

Phase	Task	Start	Finish	Duration	Time(h)	
1	Project Initiation and Research	2025/07/14	2025/09/14	63 days		
	Identify the research problem	2025/07/14	2025/08/09	27 days	35	
	Define project scope and objectives	2025/07/14	2025/08/16	34 days	60	
	Study the domain area	2025/07/14	2025/09/14	63 days	225	600
	Literature review	2025/07/14	2025/09/14	63 days	230	
	Project proposal	2025/08/10	2025/09/14	36 days	50	
2	System Architecture Design	2025/08/31	2025/09/20	21 days		
	Define system architecture	2025/08/31	2025/09/13	14 days	40	60
	Setup development environment	2025/09/07	2025/09/20	14 days	20	
3	Prototype Implementation	2025/09/21	2025/12/19	90 days		
	Implement Ethereum contracts	2025/09/21	2025/10/11	21 days	90	
	Implement the sequencer	2025/09/21	2025/10/18	28 days	160	
	Implement the executor	2025/10/05	2025/11/01	28 days	180	
	Implement the prover	2025/10/19	2025/11/22	35 days	250	935
	Implement the submitter and DA modes	2025/11/09	2025/12/06	28 days	125	
	Prototype integration and testing	2025/09/28	2025/12/06	70 days	90	
	Deploy prototype on an Ethereum testnet	2025/11/30	2025/12/19	20 days	40	
4	Conduct Experiments, Benchmarking and Optimization	2025/12/13	2026/02/01	51 days		
	Establish benchmarking metrics	2025/12/13	2025/12/19	7 days	10	
	Develop benchmarking scripts	2025/12/13	2025/12/26	14 days	45	
	Conduct controlled experiments	2025/12/20	2026/02/01	44 days	120	350
	Benchmarking data collection	2025/12/20	2026/02/01	44 days	25	
	Optimization	2025/12/20	2026/02/01	44 days	150	
5	Data Analysis	2026/01/26	2026/02/08	14 days		
	Perform statistical analysis	2026/01/26	2026/02/08	14 days	60	80
	Validate findings	2026/02/02	2026/02/08	7 days	20	
6	Documentation, Reports and Publications	2026/02/09	2026/05/08	89 days		
	Documentation of the prototype	2026/02/09	2026/02/15	7 days	30	
	Prepare research paper and publications	2026/02/16	2026/04/26	70 days	100	215
	Prepare final report	2026/02/23	2026/04/26	63 days	70	
	Final presentation	2026/04/27	2026/05/08	12 days	15	
	Total Workload					**2240**

Figure 5: Workload calculation for the project.

The total workload is distributed across the four members, with each member contributing approximately 560 hours over the project duration. The approximate time allocation per member for each phase is detailed below.

Phase	**Harshana**	**Malindu**	**Lishan**	**Shalindra**
Research	150	150	150	150
System Architecture Design	15	25	10	10
Implementation	245	220	235	235
Experiments, Benchmarking and Optimization	80	90	90	90
Data Analysis	15	20	20	25
Documentation, Reports and Publications	55	55	55	50
Total Workload(h)	**560**	**560**	**560**	**560**

Table 3: Estimated workload distribution per team member.

5 Feasibility and Conclusion

5.1 Feasibility

The proposed project to develop and benchmark a modular ZK-Rollup framework for improving Ethereum's scalability is both technically and practically feasible, given the resources, timeline, and existing technologies. The feasibility is assessed across technical, resource, and time dimensions, with considerations for potential risks and mitigation strategies.

5.1.1 Technical Feasibility

The project leverages established tools and frameworks, such as zkSync and Polygon zkEVM, for core ZK-Rollup components, ensuring a robust foundation. The implementation uses well-documented technologies:

- **Smart Contracts**: Solidity is a mature language for Ethereum, with libraries like OpenZeppelin providing reusable components for `RollupBridge.sol` and `Verifier.sol`.
- **Off-chain Sequencer**: Node.js/TypeScript is widely used for scalable off-chain systems, and Sparse Merkle Trees are standard for state management in ZK-Rollups.
- **Proof Systems**: Circom with Groth16/Plonk is production-ready for zk-SNARKs, with Plonky2 and Halo2 as viable alternatives for comparative experiments. Existing libraries reduce development overhead.
- **Data Availability**: EIP-4844 (proto-danksharding) is integrated into Ethereum testnets (e.g., Sepolia), and off-chain solutions like IPFS are accessible for prototyping.

The modular architecture ensures flexibility, allowing integration of different proof systems or transaction types (e.g., ERC20, ERC721). Deployment on an Ethereum testnet is straightforward, as testnets like Sepolia or Goerli provide free resources for experimentation. The use of synthetic workload generators and standardized benchmarking scripts aligns with industry practices, ensuring reliable metrics collection.

5.1.2 Resource Availability

The project is designed for a final-year research, requiring resources accessible to a student:

- **Computational Resources**:

Phase	CPU	RAM	GPU	Storage
Dev/Testing	4 cores	16 GB	None	250 GB SSD
Prototype Bench	8-16 cores	32-64 GB	RTX 3080 / A6000	1 TB SSD

Table 4: System requirements for different phases of ZK-Rollup prototype development and benchmarking.

- **Software Tools**: Open-source tools (Circom, Hardhat, Node.js, Docker) are freely available. Benchmarking scripts and Jupyter notebooks for analysis require no additional licensing costs.

- **Knowledge and Skills**: The project assumes proficiency in Solidity, JavaScript/TypeScript, and basic cryptography (e.g., zk-SNARKs). Online resources, such as zkSync documentation and Ethereum developer guides, provide sufficient support for learning and troubleshooting.

Access to academic resources (e.g., IEEE papers, arXiv) and open-source communities (e.g., GitHub, Ethereum forums) further supports implementation and validation. No specialized hardware (e.g., GPUs for proof generation) is required for the prototype scale, as Circom and Groth16 are optimized for standard hardware.

5.1.3 Time Feasibility

The proposed timeline (July 2025 to April 2026, approximately 10 months) is realistic for the project. Key tasks, architecture design, prototype implementation, benchmarking, optimization experiments, analysis, and reporting are structured sequentially with buffers for delays. The modular design allows parallel work (e.g., data availability integration alongside testnet deployment) to optimize time. Potential delays, such as debugging proof generation or testnet instability, are mitigated by using established frameworks and allocating extra time in critical phases like implementation and experimentation.

5.1.4 Risks and Mitigation

Potential risks include:
- **Complexity of ZK Proofs**: Circuit design and proof generation may be time-intensive. Mitigation: Start with simple circuits (e.g., token transfers) and use pre-built libraries like Circom.
- **Testnet Issues**: Network congestion or testnet upgrades could disrupt deployment. Mitigation: Use multiple testnets (e.g., Sepolia, Goerli) and local test environments (e.g., Hardhat).
- **Data Analysis Challenges**: Statistical analysis may reveal unexpected results. Mitigation: Use robust tools (e.g., Python with pandas, Jupyter) and repeat experiments for statistical significance.
- **Sequencer Performance**: The choice of Node.js/TypeScript for the sequencer, while practical for prototyping, may present performance limitations due to its single-threaded nature. Mitigation: This is acceptable for a prototype, but we acknowledge that a production-grade system would likely require a more performant language like Go or Rust.

The project's focus on modularity and reproducibility ensures that partial outcomes (e.g., a working prototype or baseline benchmarks) are valuable even if experiments face delays.

5.1.5 Societal and Economic Feasibility

There is a significant and growing demand for scalable blockchain solutions across industries. The forecasted business value of blockchain (over $3.1 trillion by 2030) and the high percentage of companies planning to integrate it underscore the economic incentive. Addressing scalability is crucial for blockchain to transition from experimental stages to mainstream production, enabling adoption in decentralised finance (DeFi), supply chain, healthcare, and IoT. This validates both the societal and economic relevance of the research.

5.2 Limitations

This project has several limitations that should be acknowledged. The prototype will be a proof-of-concept and not a production-ready system. It will only support a limited set of simple transactions (token transfers, deposits, withdrawals) and will not be EVM-compatible, which means it cannot run general-purpose smart contracts. The use of synthetic workloads, while based on real-world traces, may not capture the full complexity and unpredictability of live network traffic. Finally, while we will use a local testnet for controlled experiments, the benchmarks on the public testnet will be subject to network congestion and other external factors that are beyond our control.

5.3 Conclusion

This project proposes the design, implementation, and evaluation of a modular ZK-Rollup prototype on an Ethereum testnet, targeting scalable token transfers with configurable batching, sequencing, and data availability strategies. By combining an off-chain sequencer, state manager, prover module, and on-chain verification, the prototype provides a research-ready framework capable of exploring trade-offs between throughput, latency, gas costs, and storage. Through systematic benchmarking and optimization experiments, including variations in batch size, frequency, sequencing policies, proof systems, and data availability compression, the project aims to generate empirical evidence validating or challenging theoretical claims about ZK-Rollup scalability. The outcomes will provide a clear understanding of the practical performance and limitations of current ZK-Rollup designs. Moreover, the modular architecture ensures extensibility, allowing future experimentation with more complex transaction types, multi-sequencer setups, or production-grade zkEVM implementations. By open-sourcing our framework and findings, we hope to provide a valuable resource for the ZK-Rollup community and to inspire further research into the practical challenges of blockchain scalability.

This project proposes the design, implementation, and evaluation of a modular ZK-Rollup prototype on an Ethereum testnet, targeting scalable token transfers with configurable batching, sequencing, and data availability strategies. By combining an off-chain sequencer, state manager, prover module, and on-chain verification, the prototype provides a research-ready framework capable of exploring trade-offs between throughput, latency, gas costs, and storage. Through systematic benchmarking and optimization experiments, including variations in batch size, frequency, sequencing policies, proof systems, and data availability compression, the project aims to generate empirical evidence validating or challenging theoretical claims about ZK-Rollup scalability. The outcomes will provide a clear understanding of the practical performance and limitations of current ZK-Rollup designs. Moreover, the modular architecture ensures extensibility, allowing future experimentation with more complex transaction types, multi-sequencer setups, or production-grade zkEVM implementations. By open-sourcing our framework and findings, we aim to provide a valuable resource for the ZK-Rollup community and to inspire further research into the practical challenges of blockchain scalability.

5.4 Future Work

While this project establishes a foundational framework, several exciting avenues for future work exist. The modular design of the prototype could be extended to support more complex operations, such as general-purpose smart contracts, by integrating a zkEVM. Further research

could also explore more advanced decentralized or shared sequencer designs to enhance censorship resistance and liveness. Finally, the framework could be used to investigate the economic and game-theoretic implications of different fee mechanisms, providing valuable insights for the design of sustainable L2 ecosystems.

References

[1] A. I. Sanka and R. C. Cheung, "A systematic review of blockchain scalability: Issues, solutions, analysis and future research," *Journal of Network and Computer Applications*, vol. 195, p. 103232, 2021.

[2] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," https://bitcoin.org/bitcoin.pdf, 2008.

[3] V. Buterin, "Ethereum: A next-generation smart contract and decentralized application platform," https://ethereum.org/en/whitepaper/, 2014.

[4] S-Logix, "Research topics for scalability issues in blockchain technology," https://slogix.in/blockchain-technology/scalability-issues-in-blockchain-technology/, 2025, accessed: Aug. 7, 2025.

[5] "Analyzing performance bottlenecks in zero-knowledge proof based rollups on ethereum," https://arxiv.org/abs/2503.22709, 2025, accessed: Aug. 23, 2025.

[6] T. A. Alghamdi, N. Javaid, and R. Khalid, "A survey of blockchain based systems: Scalability issues and solutions, applications and future challenges," 2024, iEEE Senior Member.

[7] A. Aldoubaee, N. H. Hassan, and F. A. Rahim, "A systematic review on blockchain scalability," 2024.

[8] Q. Zhou, H. Huang, Z. Zheng, and J. Bian, "Solutions to scalability of blockchain: A survey," *IEEE Access*, vol. 8, pp. 16 440–16 455, 2020.

[9] "Ethereum chain overview — tps and activity," https://chainspect.app/chain/ethereum, accessed Sep. 9, 2025.

[10] "Tps across blockchains — ethereum row," https://chainspect.app/sorting/tps, accessed Sep. 9, 2025.

[11] "Ethereum average transaction fee (usd)," https://ycharts.com/indicators/ethereum_average_transaction_fee, accessed Sep. 9, 2025.

[12] "Ethereum avg. transaction fee historical chart," https://bitinfocharts.com/comparison/ethereum-transactionfees.html, accessed Sep. 9, 2025.

[13] "Ethereum transactions per day," https://ycharts.com/indicators/ethereum_transactions_per_day, accessed Sep. 9, 2025.

[14] "Ethereum statistics (transactions last 24h)," https://bitinfocharts.com/ethereum/, accessed Sep. 9, 2025.

[15] V. Buterin, "Serenity design rationale," https://notes.ethereum.org/@vbuterin/serenity_design_rationale, accessed Sep. 9, 2025.

[16] "Scaling — ethereum.org developer docs," https://ethereum.org/en/developers/docs/scaling/, accessed Sep. 9, 2025.

[17] "Zero-knowledge rollups — ethereum.org," https://ethereum.org/en/developers/docs/scaling/zk-rollups/, accessed Sep. 9, 2025.

[18] "Optimistic rollups — ethereum.org," https://ethereum.org/en/developers/docs/scaling/optimistic-rollups/, accessed Sep. 9, 2025.

[19] "Ethereum dencun upgrade lowers transaction fees for l2s," https://cointelegraph.com/news/ethereum-dencun-transaction-fees-l2, accessed Sep. 9, 2025.

[20] "Ethereum layer 2s show a dramatic drop in transaction fees after dencun," https://www.theblock.co/post/282417/ethereum-layer-2s-show-dramatic-drop-in-transaction-fees-after-dencun, accessed Sep. 9, 2025.

[21] "150 days after dencun," https://www.galaxy.com/insights/research/ethereum-150-days-after-dencun, accessed Sep. 9, 2025.

[22] "L2beat — activity dashboard," https://l2beat.com/scaling/activity, accessed Sep. 9, 2025.

[23] "Base chain overview — tps and activity," https://chainspect.app/chain/base, accessed Sep. 9, 2025.

[24] M. A. Habib, "Analyzing performance bottlenecks in zero-knowledge proof based rollups on ethereum," *arXiv preprint arXiv:2503.22709*, 2025. [Online]. Available: https://arxiv.org/abs/2503.22709

[25] K. M. Gogol, S. Gurgul, F. N. Siddiqui, D. Branes, and C. J. Tessone, "Scaling defi with zk rollups: Design, deployment, and evaluation of a real-time proof-of-concept," *arXiv preprint arXiv:2506.00500*, 2025, cC BY 4.0. [Online]. Available: https://arxiv.org/abs/2506.00500

[26] D. N. Khan, L. T. Jung, and M. A. Hashmani, "Systematic literature review of challenges in blockchain scalability," *Applied Sciences*, vol. 11, no. 20, p. 9372, 2021.

[27] Ethereum Foundation, "Scaling — ethereum.org (developers)," https://ethereum.org/en/developers/docs/scaling/, 2025, accessed Sep. 9, 2025.

[28] A. Yakovenko, "Solana: A new architecture for a high-performance blockchain v0.8.13," https://solana.com/solana-whitepaper.pdf, 2018.

[29] S. Blackshear *et al.*, "Sui lutris: A blockchain combining broadcast and consensus," *arXiv preprint arXiv:2310.18042*, 2023. [Online]. Available: https://arxiv.org/abs/2310.18042

[30] Ethereum Foundation, "Optimistic rollups — ethereum.org (developers)," https://ethereum.org/en/developers/docs/scaling/optimistic-rollups/, 2025, accessed Sep. 9, 2025.

[31] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.

[32] Q. Zhou, H. Huang, Z. Zheng, and J. Bian, "Solutions to scalability of blockchain: A survey," *IEEE Access*, vol. 8, pp. 16 440–16 455, 2020.

[33] B. Chen *et al.*, "A comprehensive survey of blockchain scalability: Shaping inner-chain and inter-chain perspectives," *arXiv preprint arXiv:2409.02968*, 2024.

[34] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology—CRYPTO '87*. Springer, 1988, pp. 369–378.

[35] "Merkle tree," https://en.wikipedia.org/wiki/Merkle_tree, accessed: Sep. 20, 2025.

[36] "Introduction to merkle tree," https://www.geeksforgeeks.org/introduction-to-merkle-tree, updated May 3, 2024; Accessed: Sep. 20, 2025.

[37] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof-systems," *SIAM Journal on Computing*, vol. 18, no. 1, pp. 186–208, 1989.

[38] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Advances in Cryptology—CRYPTO '86*, 1987, pp. 186–194.

[39] J. Groth, "On the size of pairing-based non-interactive arguments," in *EUROCRYPT 2016*, 2016, pp. 305–326.

[40] A. C. Tran *et al.*, "An implementation and evaluation of layer 2 for ethereum with zk-rollup," in *Lecture Notes in Computer Science (LNCS)*, vol. 13831, 2023, pp. 107–115.

[41] K. Gogol *et al.*, "Scaling defi with zk rollups: Design, deployment, and evaluation of a real-time proof-of-concept," https://arxiv.org/abs/2506.00500, May 2025, arXiv:2506.00500 [cs.CR].

[42] M. A. Habib, "Analyzing performance bottlenecks in zero-knowledge proof based rollups on ethereum," https://arxiv.org/abs/2503.22709, Mar 2025, arXiv:2503.22709 [cs.CR].

[43] T. Lavaur *et al.*, "Modular zk-rollup on-demand," https://arxiv.org/abs/2306.02785, Jun 2023, arXiv:2306.02785 [cs.CR].

[44] S. Chaliasos, I. Reif, A. Torralba-Agell, J. Ernstberger, A. Kattis *et al.*, "Analyzing and benchmarking zk-rollups," in *6th Conference on Advances in Financial Technologies (AFT 2024)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 316, 2024, pp. 6:1–6:23. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.AFT.2024.6

[45] D. Labs, "Is the future of ethereum rollups based?" https://www.dwf-labs.com/research/453-ethereum-based-rollups-explained, Aug 2025.

[46] S. Rewards, "The first zk-rollup with a decentralized sequencer set," https://www.stakingrewards.com/journal/research/the-first-zk-rollup-with-a-decentralized-sequencer-set, Dec 2024.

[47] Cyfrin, "What are zk rollups? how zero-knowledge rollups work," https://www.cyfrin.io/blog/what-is-a-zk-rollup, Oct 2024.

[48] Bankless, "Why every ethereum l2 will become a zk rollup," https://www.bankless.com/read/why-every-ethereum-l2-will-become-a-zk-rollup, Aug 2025.

[49] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Scalable, transparent, and post-quantum secure computational integrity," https://eprint.iacr.org/2018/046.pdf, 2018, iACR ePrint 2018/046.

[50] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge," https://eprint.iacr.org/2019/953.pdf, 2019, iACR ePrint 2019/953.

[51] M. Maller *et al.*, "Sonic: Zero-knowledge snarks from linear-size universal and updatable structured reference strings," https://eprint.iacr.org/2019/099.pdf, 2019, iACR ePrint 2019/099.

[52] "Understanding the difference between zk-snarks and zk-starks," https://chain.link/education-hub/zk-snarks-vs-zk-starks, 2023, last Updated Date: November 30, 2023.

[53] S. Motepalli, L. Freitas, and B. Livshits, "Sok: Decentralized sequencers for rollups," *arXiv preprint arXiv:2310.03616*, 2023.

[54] T. Lavaur, J. Detchart, J. Lacan, and C. P. C. Chanel, "Modular zk-rollup on-demand," 2023, iSAE-SUPAERO, University of Toulouse, University Toulouse III Paul Sabatier.

[55] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum Project, Tech. Rep. 151, Apr. 2014. [Online]. Available: https://ethereum.github.io/yellowpaper/paper.pdf

[56] Y. Fu, M. Jing, J. Zhou, P. Wu, Y. Wang, L. Zhang, and C. Hu, "Quantifying the blockchain trilemma: A comparative analysis of algorand, ethereum 2.0, and beyond," *Proceedings of the IEEE*, vol. 112, no. 2, pp. 158–174, 2024.

[57] A. C. Tran, V. V. Thanh, N. C. Tran, and H. T. Nguyen, "An implementation and evaluation of layer 2 for ethereum with zk-rollup," in *Lecture Notes in Computer Science*. Springer, 2023, pp. 57–63.

[58] S. Chaliasos, I. Reif, A. Torralba-Agell, J. Ernstberger, A. Kattis, and B. Livshits, "Analyzing and benchmarking zk-rollups," 2024, https://eprint.iacr.org/2024/889.

[59] M. A. Habib, "Analyzing performance bottlenecks in zero-knowledge proof based rollups on ethereum," 2025, mhabib@tulane.edu.

[60] S. Chaliasos, N. Mohnblatt, A. Kattis, and B. Livshits, "Pricing factors and tfms for scalability-focused zk-rollups," 2024.

[61] M. B. Saif, S. Migliorini, and F. Spoto, "A survey on data availability in layer 2 blockchain rollups: Open challenges and future improvements," *Future Internet*, vol. 16, no. 9, p. 315, 2024.

[62] Matter Labs, "ZKsync Docs," https://docs.zksync.io/, 2025.

[63] StarkWare Industries, "Starknet Documentation: Index," https://docs.starknet.io/, 2025.

[64] M. I. Silva, J. Messias, and B. Livshits, "A public dataset for the zksync rollup," 2025, nOVA Information Management School, MPI-SWS, Imperial College London, Matter Labs.

[65] H. S'aez de Oc'ariz Borde, "An overview of trees in blockchain technology: Merkle trees and merkle patricia tries," University of Cambridge Preprint, 2022, https://www.researchgate.net/publication/358740207_An_Overview_of_Trees_in_Blockchain_Technology_Merkle_Trees_and_Merkle_Patricia_Tries.

[66] Celestia, "How Celestia works: Overview," https://docs.celestia.org/learn/how-celestia-works/overview, 2025.

[67] J. Stephan, M. Pavlovic, A. Locascio, and B. Livshits, "Crowdprove: Community proving for zk rollups," 2025.

[68] N. Sheybani, A. Ahmed, M. S. Sayed, M. Safaralizadeh, M. R. Ahmed, M. A. Zakeri, H. Ghasemian, M. Kinsy, and F. Koushanfar, "Zero-knowledge proof frameworks: A survey," 2025.

[69] iden3, "Circom 2 Documentation," https://docs.circom.io/, 2025.

[70] Sui Foundation, "Groth16," https://docs.sui.io/guides/developer/cryptography/groth16, 2025.

[71] Polygon Labs, "Introducing plonky2," Polygon Blog, Jan 2022, https://polygon.technology/blog/introducing-plonky2.

[72] Zcash, "The halo2 book," https://zcash.github.io/halo2/index.html, 2025.

[73] Protocol Labs, "IPFS—Content Addressed, Versioned, P2P File System," https://github.com/ipfs/papers/raw/master/ipfs-cap2pfs/ipfs-p2p-file-system.pdf, 2025.

[74] F. Vogelsteller and V. Buterin, "Erc-20: Token standard (eip-20)," Ethereum Improvement Proposals, Nov 2015, https://eips.ethereum.org/EIPS/eip-20.

[75] ——, "Erc-721: Non-fungible token standard (eip-721)," Ethereum Improvement Proposals, Jan 2018, https://eips.ethereum.org/EIPS/eip-721.